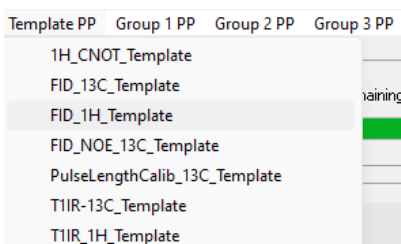


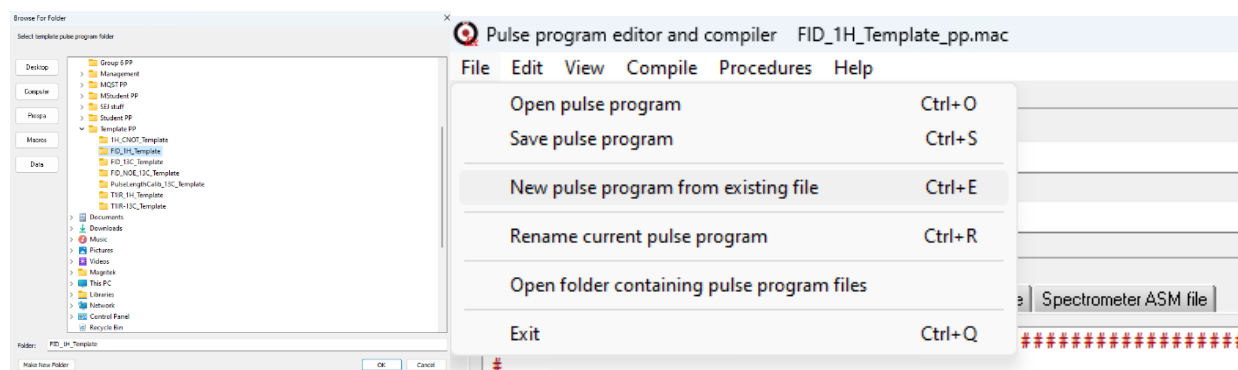
Pulse Programming - QuickStart

Introduction: The goal of this document is to provide formatting information for writing pulse programs and using Spinsolve Expert software. There will also be a section designed to discuss some quirks of the program that you should be aware of placed at the end of the document. Note this is a quick start, so it is designed to give only the key aspects, briefly. The full pulse program guide is in the “SpinsolveExpert – Pulse Programming Guide” which is provided to you.

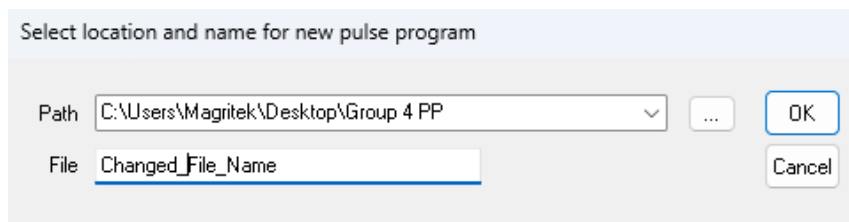
Opening, Copying, and Saving a Pulse Program File: To open a program file, hover over the desired pulse program in its pulse program tab and press “shift+left click.” This is shown for FID_1H_Template in the Template PP tab below.



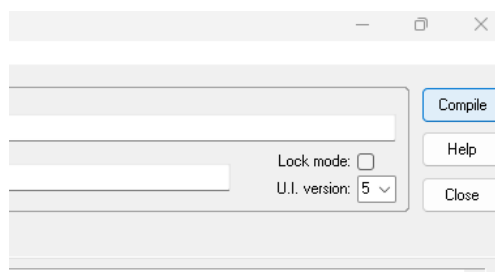
Once this is done, a window should open containing the pulse program. To make a copy of the file, go to the “file” tab and press “New pulse program from existing file” as shown below on the right or use the hot key “ctrl+E.” A window like the one on the left will come up. This should always be on the correct life if you are in the desired pulse program, so just hit “ok.”



Finally, a small window will show up like the one below. Use the drop-down menu to choose the file path leading to your group’s directory. Change the file name to what you want, but keep in mind that there cannot be any spaces. For example, it should be “File_name_here” instead of “File name here” because otherwise your program won’t compile properly.

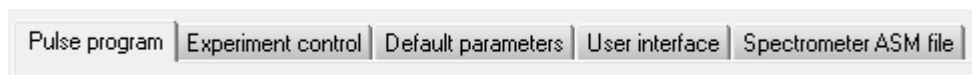


Lastly, be sure to both save, Ctrl-S, and then compile your program with the “compile” button in the top right corner otherwise it won’t run.



Whenever changes are made to your program, you must save and compile again, so be sure to do this every time you are done editing your program before exiting.

Program Interface Structure: In each experiment, there are 5 tabs: Pulse Program, Experiment Control, Default Parameters, User Interface, and Spectrometer ASM File. Note that generally, of these 5, you should only need to work in the Pulse Program tab.



The Pulse Program tab allows for users to edit the user interface inputs/order of display, define relationships of the values provided through the interface to variables used in the pulse program, and construct the pulse program sequence performed on the nuclei.

Starting with the user interface list, we can see 2 important components: the order of the parameters and the variable structure. Below, the corresponding order of the parameters in this list relative to the order in the user interface are shown.

```
interface = ["nucleus",      "Nucleus",      "tb",  "readonly_string";
             "blFreq1H",    "1H frequency (MHz)", "tb",  "freq";
             "centerFreqPPM", "Centre frequency (ppm)", "tb",  "float";
             "90Amplitude1H", "Pulse amplitude (dB)", "tb",  "pulseamp";
             "pulseLength1H", "Pulse length (us)", "tb",  "pulselength";
             "repTime",      "Repetition time (ms)", "tb",  "reptime";
```

Nucleus:	1H
1H frequency (MHz):	62.3718290
Centre frequency (ppm):	6
Pulse amplitude (dB):	0
Pulse length (us):	13
Repetition time (ms):	10000

These are ordered in the interface as they are ordered in the interface list. These have 4 parameters: the variable name, interface label, type of user interface element, and information about the entered data. While the first 2 are fairly clear in the above example: 2nd column is what the user sees and 1st column is

the variable name the input values/strings are stored in, the 3rd and 4th aren't as clear. The 3rd column has a few options:

```
tb ..... a text entry field
tm ..... a text menu (a text entry field with a pull down menu)
cb ..... a check box
rb ..... radio buttons
bt ..... a standard button
dv ..... a line with a label to divide controls into groups
```

The 4th gives the character of the variable. For the above examples, we see that there are several predefined behaviors, such as “reptime” and “pulse length” which give basic starting values in the interface of 10000ms and 13μs, respectively. This works by giving input values based on the default values stored in the system for a given nucleus. For example, the 13μs pulse length is the default 1H nucleus pulse length used for 90 degree pulses.

There is also the “freq” option, which automatically suggested the 62.3718290 MHz frequency. This is not from default values, but rather determined by the current shim values of the system. Not all variables created are this specific, though. A good way to see how these same variables might be written by the user is seen in the user interface tab, which is automatically updated when compiling changes to the pulse program tab's interface list. This breaks down the variables further:

```
interface = [
    "nucleus", "Nucleus", "tb", "readonly_string", "",
    "blFreq1H", "1H frequency (MHz)", "tb", "double", "[1,400]",
    "centerFreqPPM", "Centre frequency (ppm)", "tb", "float", "",
    "90Amplitude1H", "Pulse amplitude (dB)", "tb", "float", "[-85,0]",
    "pulseLength1H", "Pulse length (us)", "tb", "float", "[0.5,1000]",
    "repTime", "Repetition time (ms)", "tb", "float", "[1,1e+008]",
```

Now we see that, for example, while the default is 0, the actual range of value options for the 90Amplitude1H variable (aka Pulse Amplitude (dB)) is a float that allows values ranging from -85 to 0 dB. If you wanted to define a variable, say 180Amplitude1H, which is functionally the same as 90Amplitude1H, you could simply add it as shown below:

```
-----
interface = ["nucleus",          "Nucleus",          "tb",  "readonly_string";
            "blFreq1H",         "1H frequency (MHz)", "tb",  "freq";
            "centerFreqPPM",    "Centre frequency (ppm)", "tb",  "float";
            "90Amplitude1H",    "Pulse amplitude (dB)", "tb",  "pulseamp";
            "180Amplitude1H",   "180 Pulse amplitude (dB)", "tb",  "float, [-85,0]";
            "pulseLength1H",    "Pulse length (us)", "tb",  "pulselength";
            "repTime",          "Repetition time (ms)", "tb",  "reptime";
```

FID_1H history-parameters	
Nucleus:	1H
1H frequency (MHz):	62.3718289
Centre frequency (ppm):	6
Pulse amplitude (dB):	0
180 Pulse amplitude (dB):	
Pulse length (us):	13
Repetition time (ms):	10000

----- Acquisition -----

Note that if you don't see the parameter on the actual interface after you enter it on the interface list, you can select which parameters appear under Experiment -> Edit Parameter Visibility or by pressing "Ctl+shift+V" on the interface.

Relationship List: The next component of the Pulse Program tab is the relationship list. This is where the actual variables used in the pulse program are initialized. Something worth noting is that the variable names do matter depending on their application (this is also noted in the "Quirks of Spinsolve" section at the end of the document). For example, when naming a variable for use as an amplitude, the name must start with 'a', and for delays it must start with 'd.' We see below:

```
# Relationships to determine remaining variable values
relationships = ["nDataPnts = nrPnts",
                "a90Amp = 90Amplitude1H",
                "d90Dur = pulseLength1H",
                "dAcqDelay = ucsUtilities:getAcqDelay(d90Dur,shiftPoints,dwellTime)",
                "offFreq1H = (centerFreqPPM-wvPPMOffset1H)*blFreq1H",
                "Q1 = offFreq1H",
                "totPnts = nrPnts",
                "totTime = acqTime"]
```

Because of this strange convention, the variable names are assigned in this section to their respective required forms. For example, "a90Amp = 90Amplitude1H" is done to conform with this requirement. Relationships defined in the pulse program relationship list are automatically added/changed in the relationship lists across the program.

Along with the relationship list, there is also the "variables" list:

```
# These parameters will be changed between experiments
variables = [""]
```

If you need a variable to change over respective scans, this sends the variable over to the experiment control tab to be changed as specified.

Pulse Program: Lastly, we have the pulse program itself. The pulse program should be written between the "initpp(dir)" and "parList = endpp()" lines placed just after the variables list. Before describing the syntax of the pulse commands, we first need to describe the phaseList.

Phase List: This is where the phases required for every pulse are accounted for. This list is a 2D array, where the values correspond to the phases as: 0 = +x 1 = +y 2 = -x 3 = -y

Note that the last row of the phase list is always the receiver phase used during acquisition, called the acquire phase or pA. We can also see that the variables p{x} refer to the xth row of the phase list, so p1 refers to the 1st row and so on. Placing a semicolon ends the row, while using commas lets you enter the column values in each row as we see in examples further below.

While it's not especially intuitive, we are going to describe two distinct ways to specify the phases in our pulse programs. Method 1 is the typical way to do things and must be used when doing phase cycling, i.e. running multiple sequential scans with different sets of phases. In Method 2 the phases are hard-coded in the pulse program. This cannot be used with phase cycles, but can make life easier when coding sets of complex pulse sequences as we will do later in the quarter.

Method 1:

This is how phases in pulse programs are described in the SpinsolveExpert documentation, where you specify all of the phases in a phaseList array and refer to them with the variables p1, ..., pN, pA in your pulse program. It is also worth briefly explaining how the system handles phase cycling. When phase cycling is on, each scan in the experiment uses the phases specified in the column used for that respective scan. For example, in the phase list below:

```
# Phase cycle list
phaseList = [0,1,2,3;    # p1 : Pulse phase
             0,1,2,3]    # pA : Acquire phase
```

1st scan will use p1 = 0 and pA = 0

2nd will use p1 = 1, pA = 1

3rd will use p1 = 2, pA = 2

4th will use p1 = 3, pA = 3

5th will loop back to the first column, so p1 = 0 and pA = 0.

This will be repeated for the total number of scans requested.

Some important words of warning that are not obvious: You must make sure that there are the same number of rows of phases as are implemented in the pulse program. For example, if the phase list has 3 rows, which assigns values for p1, p2, and pA, but in the pulse program only p1 and pA are ever used, the data acquired will contain additional and sometimes significant noise. This is simply a quirk of how the system processes the phase list and shouldn't be interpreted as something physical. Simply being careful about writing these in the correct format will save you a lot of headaches. However, if implemented correctly, phase cycling can greatly improve the signal-to-noise ratio of your data.

Method 2:

In cases that you don't need to do phase cycling, this section explains how to manage the phase list in such a way that you don't get accidental errors. The best solution maybe simply not to use it. When phase cycling is turned off (the box is unchecked), the program sets all the phase variables implied by the phaseList to 0. It does not use just the first column say, so if you use the same methodology highlighted above, you may accidentally input only +x phases for all your pulses.

In this method you create a minimal phaseList with just one line that sets the acquisition phase. Here we set it to +x so that regardless of whether phase cycling is turned on, the phases will remain the same.

```
# Phase cycle list
phaseList = [0] #pA
```

You then directly set the phases in the pulse function calls.

This would look like:

```
pulse(1, a90H, 0, d90H)
```

Instead of this:

```
pulse(1, a90H, p1, d90H)
```

Where the 3rd element lists the phase as discussed below. By directly inputting the phase we want, we bypass the phase list entirely, making this a significantly less error-prone process.

Pulse Commands:

With this out of the way, we can now move on to the pulses themselves. The most basic form of a pulse might be an $R_x(\pi/2)$ pulse on the 1H channel as seen below:

```
pulse(1, a90H, p1, d90H)
```

We see this has 4 components:

The 1 indicates the channel number. In this case, that would be pulsing on the 1H channel. a90H indicates the amplitude of the pulse being applied in dB. p1, as discussed above, indicates to use the 1st phase of the phase list. If p1 corresponds to 0 (according to the phaseList) this is a +x pulse. Lastly, the d90H indicates how long the pulse should be applied in μ s. Remember you can also simply directly input 0 into this place if using method 2, as highlighted above.

Now suppose we have 2 channels active: 1 for 1H and 1 for 13C, as we will for labs 2-4. Channel 1 is 1H and channel 2 is 13C. We now want to perform a $R_y(-\pi/2)$ pulse on the 13C. How would this be done?

The first step would be to add a user input for pulseLength13C and 90Amplitude13C in the interface and relate these to a90C and d90C in a similar structure as discussed earlier for defining the relationships and interface variables around 1H. With this, we could then make the $R_y(-\pi/2)$ as follows (where p3 = 3):

```
pulse(2, a90C, p3, d90C)
```

Lastly for the pulse structures, we want to look at how to make pulses simultaneous on 2 different channels. Suppose we have the same coding we just did above in a program designed for the 1H and 13C channels. To do this, we need 3 extra variables. The 1st two are the frequency of the channels. This can be done by defining the variables freq1H and freq13C, which store the 1H channel frequency and 13C channel frequency, respectively. The last one is the amplitude of the pulse on 1H such that the desired pulse on the 1H nuclei occurs within the same time span as the desired pulse on 13C. We'll call this a90HC. Then, for a simultaneous pulse of $R_x(-\pi/2)$ on 1H and $R_y(\pi/2)$ on 13C, we would input:

```
pulse(1,a90HC,p4,freq1H,2,a90C,p2,freq13C,d90C)
```

Where p4 = 2 and p2 = 1.

Lastly, when performing a sequence of pulses, you must insert some delay time between them, since the spectrometers transmitter needs a brief amount of time to finish sending a pulse before the next one. Many experiments have long delays inserted already, e.g. Inversion Recovery, but not all. The “delay” function waits a specified amount of time before continuing with the next operation, generally this should be around 1 μ s. Variables entered into the delay function need to start with a ‘d’ and should be provided in units of μ s. A delay variable listed in μ s formatted properly for the delay would be of the form:

```
delay(dRepTime)
```

Also, if you need to perform a longer delay than is allowed by delay() for any reason, it can be done with the wait() function with the same restrictions in formatting as the delay() function, except that its variables need to start with a ‘w’ instead of a ‘d’, as you should now expect.

Quirks of Spinsolve:

In this section, we’ll just list some odd behavior in the programming structure that may cause some issues if you aren’t careful. Some of these are mentioned above, but this is here for quick reference.

- Amplitude variables need to start with an ‘a’
- Pulse length and Delay variables need to start with a ‘d’
- Wait variables need to start with a ‘w’
- If not all rows in the phase list are used, we will get an error in the system that appears as noise. This is not physical, but rather just a bug in the software. Make sure you use all variables when using the phase list, as discussed in method 1 of the phase cycling section.
- Turning off the phase cycling sets all values of the phase list to zero. If you need to run a program with phase cycling off, refer to method 2 in the phase cycling section.
- Changing the relationship list and/or the interface list in the pulse program tab changes the respective lists in all other tabs after compiling. However, this isn’t true in reverse. For example, changing the interface list in the user interface tab should not be done as it will not save properly.
- When making a new pulse program from a template, the name must not have spaces, but rather “_” are good for replacements. For example, a file name should be “1H_FID” instead of “1H FID” for it to run.
- If a pulse program uses the “acquireon” and “acquireoff” functions, do not run it without having both in the program, or this may lead to the program not ending. This also can damage the Spinsolve system in some cases.